

Hibernate - Query Language

- Hibernate Query Language (HQL) is an object-oriented query language used in Hibernate to interact with the database. Unlike SQL, which works directly with tables and columns, HQL works with persistent objects (entities) and their properties. HQL is database-independent, meaning queries are written in terms of entity classes and attributes and Hibernate automatically translates them into the appropriate SQL for the underlying database.

Key Features of HQL

- **Object-Oriented:** Operates on entity objects instead of database tables.
- **Portable:** Database-independent same HQL query works on different databases.
- **Powerful Queries:** Supports joins, aggregations, grouping, ordering, subqueries, etc.
- **Polymorphic Queries:** Supports inheritance queries can work on parent classes and include child class entities.
- **Automatic Mapping:** Properties in HQL map directly to Java class fields, not column names.

The Query interface provides object-oriented methods and capabilities for representing and manipulating HQL queries.

Note:

- *Keywords like FROM, SELECT and WHERE are not case-sensitive in HQL.*
- *Table and Column names are case-sensitive in HQL.*

Hibernate Query Language (HQL) Clauses

There are many HQL clauses available to interact with relational databases and several of them are listed below:

1. FROM Clause
2. SELECT Clause
3. WHERE Clause

4. ORDER BY Clause
5. UPDATE Clause
6. DELETE Clause
7. INSERT Clause

For details and examples regarding each clause is mentioned below.

1. FROM Clause

To load a whole persistent object into memory, the FROM clause is used.

```
String hib = "FROM Student";  
Query query = session.createQuery(hib);  
List results = query.list();
```

2. SELECT Clause

The SELECT clause is used when only a *few attributes of an object* are required rather than the entire object.

```
String hib = "SELECT S.roll FROM Student S";  
Query query = session.createQuery(hib);  
List results = query.list();
```

3. WHERE Clause

Filtering records is done with the WHERE clause. It's used to *retrieve only the records that meet a set of criteria*.

```
String hib = "FROM Student S WHERE S.id = 5";  
Query query = session.createQuery(hib);  
List results = query.list();
```

4. ORDER BY Clause

The ORDER BY clause is used to *sort the results* of an HQL query.

```
String hib = "FROM Student S WHERE S.id > 5 ORDER BY S.id DESC";  
Query query = session.createQuery(hib);  
List results = query.list();
```

NOTE:

1. **Order By - DESC** will sort in descending order
2. **Order By - ASC** will sort in ascending order

5. UPDATE Clause

The UPDATE clause is required to *update the value* of an attribute.

```
String hib = "UPDATE Student set name=:n WHERE roll=:i";  
Query q=session.createQuery(hib);  
q.setParameter("n", "John");  
q.setParameter("i", 23);  
int status=q.executeUpdate();  
System.out.println(status);
```

6. DELETE Clause

It is required to *delete a value* of an attribute.

```
String hib = "DELETE FROM Student WHERE id=10";  
Query query=session.createQuery(hib);  
query.executeUpdate();
```

7. INSERT Clause

It is required to *Insert values* into the relation.

```
String hib = "INSERT INTO Student(first_name, last_name)" +  
"SELECT first_name, last_name FROM backup_student";  
Query query = session.createQuery(hib);  
int result = query.executeUpdate();
```

Pagination using Query

For pagination using query we have **two** methods available for it, below table contains the methods and its description.

1. **Query setMaxResults(int max):** Instructs Hibernate to get a specific number of items.
2. **Query setFirstResult(int starting_no):** Takes an integer as an argument that represents the first row in your result set, beginning with row 0.

Example for pagination using HQL

To fetch the few row data at a time we can use pagination using HQL. For this example we are fetching the rows 5 to 10.

```
String hib = "FROM Student"  
Query query=session.createQuery(hib);  
query.setFirstResult(5);  
query.setMaxResult(10);  
List list=query.list();
```

The above example returns the list of records from 5 to 10.

Aggregate Methods

Similar to SQL, HQL has several aggregation techniques, some of them are mentioned below-

- Average
- Max
- Min
- Count

- Sum

For details and examples regarding each aggregate method is mentioned below.

Average

To find the average of *marks*

```
String hib = "SELECT AVG(marks) FROM Student";  
Query q=session.createQuery(hib);
```

Max

To find the maximum *mark* among all the available marks.

```
String hib = "SELECT MAX(marks) FROM Student";  
Query q=session.createQuery(hib);
```

Min

To find the minimum *mark* among all the available marks.

```
String hib = "SELECT MIN(marks) FROM Student";  
Query q=session.createQuery(hib);
```

Count

To find the count of *id* present.

```
String hib = "SELECT COUNT(id) FROM Student";  
Query q=session.createQuery(hib);
```

Sum

To find the sum of all the marks of Student.

```
String hib = "SELECT SUM(marks) FROM Student";  
Query q=session.createQuery(hib);  
List<Integer> list=q.list();  
System.out.println(list.get(0));
```